

RAG-LLM Project Summary (Presentation + Review)

วันที่สรุป: 2026-04-20

Workspace: rag-llm

1) Executive Summary

โปรเจกต์นี้คือระบบ RAG-LLM สำหรับเอกสาร PDF ที่มีทั้งฝั่ง Backend (FastAPI + LlamaIndex + Ollama + ChromaDB) และ Frontend (React + Vite + Zustand) โดยภาพรวมที่ทำไว้แล้วมี 4 แกนหลัก:

1. Ingestion Pipeline: อัปโหลด PDF, สกัดข้อความ, ทำ OCR fallback, สร้าง index
2. Retrieval + Generation Pipeline: Hybrid Search (Vector + BM25), rerank ด้วย FlashRank, ดอนพร้อม citation
3. Product UX: Workspace แชนแนลเอกสาร, Model Compare, Knowledge Actions (mindmap/chart/slides/infographic), PDF preview
4. Runtime Control: สลับ CPU/GPU runtime, รอ pending requests, warmup โมเดล, polling สถานะ restart

2) สิ่งทีระบบทำได้ในปัจจุบัน (What Has Been Built)

Backend

- API สำหรับ upload, status polling, chat single, suggest title, compare, knowledge action, runtime management
- แยกข้อมูลตาม session ด้วย collection ต่อ session
- สร้าง citation อ้างอิงชื่อไฟล์และเลขหน้า
- รองรับเอกสารไทย/อังกฤษผ่าน OCR fallback

Frontend

- หน้า Workspace: Upload + Chat + Knowledge Panel + Compare-in-Workspace
- หน้า Model Arena: ถามคำถามเปรียบเทียบ 2 โมเดล
- หน้า Settings: สลับ runtime CPU/GPU พร้อมสถานะ progress
- State persistence ด้วย Zustand middleware (session, document, chat history)

3) Architecture และ Flow หลัก

3.1 Ingestion Flow (Upload -> Index -> Ready/Completed)

1. Frontend ส่ง POST /upload พร้อม file และ session_id
2. Backend validate เป็น PDF และบันทึกไฟล์ลง uploaded_docs/{session}_{filename}
3. Backend สั่ง background task: document_processor.process_document(...)
4. process_document สกัดข้อความรายหน้า (PyPDF ก่อน, OCR ถ้าข้อความน้อย)
5. สร้าง Vector Index ต่อ session + BM25 retriever ภาษาไทย
6. สถานะเปลี่ยนเป็น ready_for_chat (ผู้ใช้เริ่มถามได้ทันที)
7. ระบบสร้าง summary ต่อใน background แล้วเปลี่ยนเป็น completed
8. Frontend polling GET /upload/status/{session_id}/{filename} ทุก 3 วินาทีเพื่อนำผลมาแสดง

3.2 Query Flow (Chat/Compare/Action)

1. API รับ query + model + session
2. สร้าง retriever แบบ hybrid: Vector + BM25
3. rerank ด้วย FlashRank (cross-encoder)
4. ส่ง context เข้า LLM ผ่าน prompt template ที่บังคับอ้างอิงหน้า
5. แยก thinking block ออกจากคำตอบ (ถ้ามี <think>...</think>)
6. สร้าง citations จาก source nodes
7. ส่ง response กลับ frontend เพื่อ render คำตอบ + citation click-to-open-PDF

3.3 Runtime Flow (CPU/GPU Switching)

1. Frontend หน้า Settings เรียก PUT /runtime/device
2. Backend ตรวจ active requests
3. ถ้าเปิด wait_for_pending จะรอให้ request ค้างจบก่อน
4. เปลี่ยน runtime global ผ่าน runtime manager
5. ล้าง LLM cache + warmup runner แบบ background
6. Frontend polling GET /runtime/restart-status เพื่ออัปเดต progress UI

4) API Catalog: ฟังก์ชันแต่ละตัวทำอะไร + หลักการทำงาน + ใช้น้ำไหน

>หมายเหตุ: ด้านล่างคือ mapping ตามโค้ดจริงทั้ง Backend และ Frontend

4.1 POST /upload -> upload_document(...)

- เป้าหมาย:

- รับไฟล์ PDF และเริ่มกระบวนการ indexing
- รับค่า:
 - multipart: file, session_id
- ส่งกลับ:
 - UploadResponse (status, filename, session_id, message)
- หลักการทำงาน:
 1. เช็คว่าไฟล์ลงท้าย .pdf
 2. บันทึกไฟล์ลง disk
 3. สั่ง background task document_processor.process_document
 4. ดอกลับทันที (ไม่ว่า process ทั้งหมด)
- Frontend ที่ใช้:
 - uploadDocument(...) ใน service
 - เรียกจาก useUpload -> ใช้งานผ่าน UploadZone ในหน้า Workspace

4.2 GET /upload/status/{session_id}/{filename} -> get_document_status(...)

- เป้าหมาย:
 - ให้ frontend polling ความคืบหน้าประมวลผลเอกสาร
- ส่งกลับ:
 - status: processing | ready_for_chat | completed | error | not_found
 - พร้อม summary/mindmap/message ตามสถานะ
- หลักการทำงาน:
 1. lookup key {session_id}_{filename} จาก in-memory status map
 2. คืน default not_found หากไม่พบ
- Frontend ที่ใช้:
 - checkDocumentStatus(...) ใน service
 - เรียกทุก 3 วินาทีใน useUpload

4.3 POST /chat/single -> chat_single(...)

- เป้าหมาย:
 - ตอบคำถามจากเอกสารใน session เดียว
- รับค่า:
 - query, model_name, session_id
- ส่งกลับ:
 - ChatResponse (thinking, answer, citations, model_name)

- หลักการทำงาน:
 1. register active request
 2. เช็ค client disconnect
 3. เรียก llm_service.query_with_context(...)
 4. คืนคำตอบ + citations
 5. unregister request ใน finally
- Frontend ที่ใช้:
 - chatSingle(...) ใน service
 - เรียกจาก useChat.sendMessage ในหน้า Workspace

4.4 POST /chat/suggest-title -> suggest_title(...)

- เป้าหมาย:
 - สร้างชื่อแชทอัตโนมัติจากข้อความแรกของผู้ใช้
- รับค่า:
 - query, optional model_name
- ส่งกลับ:
 - { title: string }
- หลักการทำงาน:
 1. สร้าง prompt ให้ตอบข้อสั้น 1-3 คำ
 2. เรียก LLM แบบ complete
 3. cleanup think tags/newline และตัดความยาว
 4. fallback เป็น query[:30] ถ้า error
- Frontend ที่ใช้:
 - suggestTitle(...) ใน service
 - เรียกหลัง first message ใน useChat

4.5 POST /chat/compare -> chat_compare(...)

- เป้าหมาย:
 - ยิง query เดียวกันไป 2 โมเดลแล้วส่งกลับเปรียบเทียบ
- รับค่า:
 - query, model_a, model_b, session_id
- ส่งกลับ:
 - CompareResponse (response_a, response_b)
- หลักการทำงาน:

1. validate ห้ามโมเดลซ้ำ
 2. register request + เช็ค disconnect
 3. ยิง query 2 โมเดลพร้อมกันด้วย asyncio.gather
 4. คืน ChatResponse ทั้ง 2 ผัง
 5. unregister request
- Frontend ที่ใช้:
 - chatCompare(...) ใน service
 - หน้า Workspace compare mode (CompareLayout) ใช้ถูกต้อง
 - หน้า Model Arena (useArenaChat) มีจุดที่ต้อง recheck เรื่อง signature

4.6 POST /actions/generate -> generate_action(...)

- เป้าหมาย:
- สร้าง output เฉพาะทางจากเอกสาร: mindmap, chart, slides, infographic
- รับค่า:
- session_id, action_type, optional model_name, user_goal, language
- ส่งกลับ:
- ActionGenerateResponse (prompt, answer, thinking, citations)
- หลักการทำงาน:
- 1. _build_action_prompt(...) เลือก prompt ตาม ActionType
- 2. query ผ่าน llm_service.query_with_context(...)
- 3. ถ้าเป็น mindmap: parse markdown เป็น mindmap JSON
- Frontend ที่ใช้:
- generateKnowledgeAction(...) ใน service
- เรียกจาก KnowledgeTabs ใน Workspace (panel ขวา)

4.7 GET /runtime/status -> get_runtime_status(...)

- เป้าหมาย:
- คืน runtime ปัจจุบัน (cpu/gpu) และจำนวน active requests
- Frontend ที่ใช้:
- getRuntimeStatus(...) ใน service
- เรียกจาก runtimeStore.fetchRuntime() บนหน้า Settings ตอน mount

4.8 PUT /runtime/device -> update_runtime_device(...)

- เป้าหมาย:

- สลับ runtime แบบ global พร้อมกลไกรอ request ดัง
- รับค่า:
 - device, optional model_names, wait_for_pending, force
- ส่งกลับ:
 - runtime status ล่าสุด
- หลักการทำงาน:
 1. ถ้ามี active requests และเปิด wait -> รอ pending complete
 2. set runtime
 3. clear LLM cache
 4. background warmup (sync_ollama_runners)
- Frontend ที่ใช้:
 - setRuntimeDevice(...) ใน service
 - เรียกจาก runtimeStore.updateRuntime(...)
 - ปุ่มในหน้า Settings

4.9 GET /runtime/restart-status -> get_restart_status(...)

- เป้าหมาย:
 - ให้ frontend polling progress ของ restart/switch
- Frontend ที่ใช้:
 - getRestartStatus(...) ใน service
 - เรียกจาก runtimeStore.startPollingRestartStatus(...)

4.10 POST /runtime/restart -> restart_backend(...)

- เป้าหมาย:
 - restart backend process แบบเต็มระบบ (optional runtime config)
- รับค่า:
 - optional device, model_names
- หลักการทำงาน:
 1. รอ pending requests (best-effort)
 2. เขียน restart_config.json
 3. spawn process ใหม่ด้วย run.py
 4. exit process เก่า
- Frontend ที่ใช้:
 - restartBackend(...) ผ่าน runtimeStore.triggerRestart(...)

- ณ โค้ดปัจจุบัน: ยังไม่พบปุ่มเรียกใช้งานจริงๆ บน Settings UI

4.11 Utility Endpoints/Path

- GET / และ GET /health
- health/information endpoint (ยังไม่พบการเรียกจาก frontend)
- Static files /docs/...
- ใช้เปิด PDF ใน PdfViewerModal บนหน้า Workspace

5) Frontend Page-to-API Mapping (พร้อมตำแหน่งใช้งาน)

5.1 Landing (/)

- ไม่มีการเรียก backend โดยตรง
- ใช้ local persisted history + session management

5.2 Workspace (/chat/:sessionId)

- Upload panel:
- POST /upload
- GET /upload/status/{session_id}/{filename}
- Chat panel:
- POST /chat/single
- POST /chat/suggest-title
- Compare mode ใน Workspace:
- POST /chat/compare
- Knowledge actions panel:
- POST /actions/generate
- PDF preview modal:
- โหลดไฟล์จาก static path /docs/{session}_{filename}

5.3 Model Arena (/arena)

- ใช้ POST /chat/compare
- มีจุดที่ต้อง recheck ใน hook การส่งพารามิเตอร์ (ดูหัวข้อ Recheck)

5.4 Settings (/settings)

- Runtime status/switch:
- GET /runtime/status

- PUT /runtime/device
- GET /runtime/restart-status
- POST /runtime/restart (ผ่าน store action, ยังไม่เห็น trigger UI ชัดเจน)

6) เทคนิคที่ใช้ในโปรเจกต์ และหลักการทำงาน

6.1 Session-Isolated Vector Store

- ใช้ collection ชื่อ chat_{session_id} ต่อ session
- ประโยชน์:
- แยก context คนละแชตชัดเจน
- ลดการปนกันของเอกสารข้าม session

6.2 OCR Fallback Strategy

- อ่านด้วย PyPDF ก่อน
- ถ้าข้อความรวมสั้นกว่า threshold (MIN_TEXT_LENGTH) ให้ fallback OCR (Tesseract)
- ประโยชน์:
- รองรับไฟล์สแกน/รูปภาพ
- เพิ่มโอกาสดึงเนื้อหาไทย-อังกฤษจากเอกสารจริง

6.3 Hybrid Retrieval (Vector + BM25)

- Vector retrieval: semantic similarity
- BM25: keyword matching โดยใช้ tokenizer ภาษาไทย
- รวมด้วย QueryFusionRetriever
- ประโยชน์:
- สมดุลระหว่าง semantic และ exact keyword
- ดีขึ้นกับเอกสารไทยที่คำเฉพาะ/ศัพท์ตรงตัวสำคัญ

6.4 FlashRank Reranking

- ใช้ cross-encoder rerank candidate nodes
- เลือก top-N หลังประเมิน relevance รอบสอง
- ประโยชน์:
- เพิ่ม precision ก่อนส่ง context เข้า LLM
- ลด noise ใน context

6.5 Citation-aware Prompting

- prompt บังคับให้แนบการอ้างอิงหน้า [หน้า X]
- response ถูก parse source nodes เป็น citation object ส่งให้ frontend
- ประโยชน์:
- auditability สูงขึ้น
- UX เชื่อมต่อกับ PDF preview ได้ตรงหน้า

6.6 Progressive UX: Ready-for-Chat ก่อน Summary เสร็จ

- ระหว่าง background processing ระบบส่งสถานะ ready_for_chat
- ผู้ใช้เริ่มถามได้โดยไม่ต้องรอ summary เสร็จ 100%
- ประโยชน์:
- ลด perceived latency
- ประสบการณ์ใช้งานดีขึ้น

6.7 Runtime Governance + Safe Switching

- RuntimeManager ติดตาม active requests
- รองรับ wait/force policy ตอนสลับ runtime
- ล้าง cache + warmup Ollama runners เพื่อให้ request ถัดไปเสถียรขึ้น

6.8 Auto Fallback GPU -> CPU

- ถ้าเจอ memory/runtime errors กลุ่ม GPU
- สลับเป็น CPU อัตโนมัติและ retry 1 ครั้ง
- ประโยชน์:
- ระบบตอบโต้ได้แม้ GPU มีปัญหา

6.9 Frontend State Persistence

- Zustand + persist middleware
- เก็บ session/chat/documents/history ใน local storage
- ประโยชน์:
- รีเฟรชหน้าแล้วยังกลับมาจุดเดิมได้

6.10 API Interceptors (Frontend)

- axios request/response/error interceptors
- normalize error message ให้ UI ใช้งานง่าย
- ใส่ header สำหรับ ngrok warning bypass

7) Recheck Summary (จุดที่ควรตรวจซ้ำ/แก้ก่อนเดโมใหญ่)

7.1 High Priority

1. ModelArena compare call signature ผิดรูปแบบ

- อาการ:
- มีการเรียก chatCompare(query, [arenaModelA, arenaModelB], sessionId)
- แต่ service นิยามเป็น (query, modelA, modelB, sessionId)
- ผลกระทบ:
- payload ไป backend ผิด field mapping และอาจตอบผิด/ล้มเหลว
- ตำแหน่ง:
- frontend/src/hooks/useChat.js
- frontend/src/services/api.js

2. ใช้ config key ที่ยังไม่ได้ประกาศ (CPU_SIMILARITY_TOP_K)

- อาการ:
- ใน llm_service.query_with_context อ้างถึง settings.CPU_SIMILARITY_TOP_K
- แต่ config.py ไม่มี field นี้
- ผลกระทบ:
- เสี่ยงเกิด runtime error เมื่อเข้าเงื่อนไข CPU mode
- ตำแหน่ง:
- backend/app/services/llm_service.py
- backend/app/config.py

7.2 Medium Priority

3. เอกสาร API/spec ยังมี drift จากโค้ดจริง

- ตัวอย่าง:
- docs บางไฟล์อ้าง endpoint/รูปแบบเก่า (/document/status, รูปแบบ compare แบบเก่า)
- ผลกระทบ:
- ทีมใหม่/ผู้ฟรีเซนต์อาจสื่อสาร API ผิด
- ตำแหน่ง:
- frontend/docs/api-specs.md

4. README backend ยังมีเนื้อหา Qdrant ทั้งที่โค้ดใช้ Chroma

- ผลกระทบ:
- onboarding สับสน

- ตำแหน่ง:
- backend/README.md

7.3 Low Priority

5. Request logging พิมพ์ headers ทั้งหมด

- ผลกระทบ:
- ถ้าโปรดักชันควร mask/sanitize header สำคัญ
- ตำแหน่ง:
- backend/app/main.py

8) สคริปต์ฟรีเชนต์แนะนำ (Talking Points)

1. Problem:

- ผู้ใช้ต้องการคุยกับเอกสารไทย/อังกฤษแบบอ้างอิงแหล่งที่มาได้จริง

2. Solution:

- RAG pipeline ที่มี OCR fallback + Hybrid retrieval + reranker + citation

3. UX Differentiator:

- Ready-for-chat ระหว่างยังสรุปเอกสารไม่เสร็จ
- Compare mode + Knowledge actions (mindmap/chart/slides/infographic)

4. Ops Readiness:

- Runtime switch CPU/GPU แบบมี safety policy
- fallback อัปเดตเมื่อเจอ GPU runtime error

5. Next hardening:

- แก้ compare signature bug
- เพิ่ม config CPU top-k
- sync docs ให้ตรงกับ implementation ปัจจุบัน

9) Quick Appendix: Endpoint Summary Table

Endpoint	Function	Purpose	Frontend Use
POST /upload	upload_document	รับ PDF + เริ่ม background indexing	Workspace Upload
GET /upload/status/{session_id}/{filename}	get_document_status	Polling สถานะเอกสาร	Workspace Upload Polling

POST /chat/single chat_single ตอบคำถามโมเดลเดี่ยวแบบ RAG Workspace Chat
POST /chat/suggest-title suggest_title สร้างชื่อแชทอัตโนมัติ Workspace first-message flow
POST /chat/compare chat_compare เปรียบเทียบคำตอบ 2 โมเดล Workspace Compare / Model Arena
POST /actions/generate generate_action สร้าง mindmap/chart/slides/infographic Workspace Knowledge Actions
GET /runtime/status get_runtime_status runtime + active requests Settings
PUT /runtime/device update_runtime_device สลับ CPU/GPU runtime Settings
GET /runtime/restart-status get_restart_status progress polling Settings runtime progress
POST /runtime/restart restart_backend restart backend process ผ่าน runtime store
GET / root app status ยังไม่เรียกจาก FE
GET /health health_check health check ยังไม่เรียกจาก FE
/docs/{session}_{filename} static mount เปิด PDF preview Workspace PDF modal

10) สรุปปิดท้าย

โปรเจกต์นี้มีแกน RAG ที่ใช้งานได้จริงและครบวงจรตั้งแต่ upload -> retrieval -> generation -> citation -> UI interaction แล้ว โดยจุดเด่นคือรองรับเอกสารไทยได้ดี (OCR + tokenizer ไทย + hybrid retrieval) และมี runtime control ที่ค่อนข้าง production-minded สำหรับงาน local LLM.

ก่อนพร็เซนต์หรือเดโมรอบสำคัญ แนะนำ fix รายการ recheck ในหัวข้อ 7 เพื่อความนิ่งของระบบและความแม่นยำของการสื่อสาร API กับทีม.